

Operadores en Lenguaje C

Estudio detallado de tipos de datos y su comportamiento

Tec. Emilio C. Vargas
Laboratorio Virtual de Programación
<https://proyectos.emiliocolor.com>

18 de septiembre de 2026

Propiedad del documento

Este documento es propiedad de **Tec. Emilio C. Vargas** y fue elaborado para el **Laboratorio Virtual de Programación** disponible en <https://proyectos.emiliocolor.com>.

Resumen

Este documento describe de forma exhaustiva el comportamiento de los distintos tipos de datos básicos del lenguaje C (**char**, **short**, **int**, **long**, **long long**, **float**, **double**, **long double**) cuando se aplican sobre ellos los operadores aritméticos, relacionales, lógicos, de bits y de asignación compuesta. Se toma como base la práctica **operadorconchar.c** y sus variantes, explicando por qué algunos operadores funcionan con ciertos tipos y con otros no, cuáles son las reglas de promoción y conversión, y cuáles son los errores típicos que aparecen al cambiar el tipo de dato de las variables **cx** y **cy**.

Índice

1. Introducción	3
2. Fundamentos: tipos de datos en C	3
2.1. Clasificación general	3
2.2. Tamaños y rangos típicos	3
3. Reglas de promoción y conversión	4
3.1. Promoción de enteros	4
3.2. Conversiones aritméticas usuales	4
3.3. Truncamiento y desbordamiento	5
4. Operadores de la práctica	5

5. Análisis por tipo de dato	5
5.1. Caso 1: char para cx y cy	5
5.2. Caso 2: short para cx y cy	6
5.3. Caso 3: int para cx y cy	6
5.4. Caso 4: long int para cx y cy	7
5.5. Caso 5: long para cx y cy	7
5.6. Caso 6: long long para cx y cy	8
5.7. Caso 7: float para cx y cy	8
5.8. Caso 8: double para cx y cy	9
5.9. Caso 9: long double para cx y cy	9
6. Tabla resumen de compatibilidad	10
7. El problema del%c y los tipos no-int	10
8. El problema del truncamiento en la práctica	11
9. Recomendaciones de codificación	11
10. Conclusión	11

1 Introducción

El lenguaje C es un lenguaje de tipado estático y relativamente débil: las variables tienen un tipo declarado, pero el compilador aplica una serie de reglas de conversión automática (promociones) cuando se combinan valores de distintos tipos. Estas reglas, junto con las restricciones propias de cada operador, determinan qué operaciones son válidas y cuáles producen errores de compilación o comportamiento indefinido.

La práctica `operadorconchar.c` aplica un conjunto completo de operadores sobre dos variables `cx` y `cy` de tipo `char`. La tabla adjunta propone repetir el ejercicio cambiando el tipo de esas dos variables por cada uno de los tipos básicos del lenguaje:

#	Código fuente	Tipo de <code>cx</code> y <code>cy</code>
1	<code>operadorconchar.c</code>	<code>char</code>
2	<code>operadorconshort.c</code>	<code>short</code>
3	<code>operadorconint.c</code>	<code>int</code>
4	<code>operadorconlongint.c</code>	<code>long int</code>
5	<code>operadorconlong.c</code>	<code>long</code>
6	<code>operadorconlonglong.c</code>	<code>long long</code>
7	<code>operadorconfloat.c</code>	<code>float</code>
8	<code>operadorcondouble.c</code>	<code>double</code>
9	<code>operadorconlongdouble.c</code>	<code>long double</code>

El objetivo de este documento es explicar, uno por uno, qué cambia al sustituir el tipo, por qué el código compila o no, y cómo se comporta en tiempo de ejecución.

2 Fundamentos: tipos de datos en C

2.1 Clasificación general

Los tipos de datos básicos de C se dividen en dos grandes familias:

- **Tipos enteros:** `char`, `short`, `int`, `long`, `long long`, y sus variantes `unsigned`.
- **Tipos de punto flotante:** `float`, `double`, `long double`.

Esta división es crucial porque **los operadores de bits y el operador módulo solo están definidos para tipos enteros**. Aplicarlos a tipos de punto flotante produce errores de compilación.

2.2 Tamaños y rangos típicos

El estándar C no fija tamaños exactos, solo garantiza mínimos. En plataformas modernas (x86-64, Linux/Windows) los tamaños habituales son:

Tipo	Bytes	Rango típico	Formato printf
char	1	−128 a 127	%c o %d
unsigned char	1	0 a 255	%c o %u
short	2	−32,768 a 32,767	%hd
unsigned short	2	0 a 65,535	%hu
int	4	$\pm 2,1 \times 10^9$	%d
unsigned int	4	0 a $4,29 \times 10^9$	%u
long	8	$\pm 9,2 \times 10^{18}$	%ld
long long	8	$\pm 9,2 \times 10^{18}$	%lld
float	4	$\approx \pm 3,4 \times 10^{38}$	%f
double	8	$\approx \pm 1,7 \times 10^{308}$	%lf
long double	16	rango extendido	%Lf

Observación importante

El tipo `long` en Linux de 64 bits ocupa 8 bytes (igual que `long long`), pero en Windows de 64 bits ocupa 4 bytes (igual que `int`). Por eso conviene no asumir tamaños y usar `sizeof` cuando sea necesario.

3 Reglas de promoción y conversión

3.1 Promoción de enteros

Cuando se realiza una operación aritmética, los operandos de tipo entero menor que `int` (`char`, `short`) se **promocionan automáticamente a int** antes de operar. Este fenómeno se llama *integer promotion*. Por ejemplo:

```

1  char a = 100, b = 50;
2  int r = a * b;    // a y b se promocionan a int, el resultado es int
3

```

Por eso, aunque `cx` y `cy` sean `char`, la multiplicación `cx * cy` se realiza realmente con `int`. El resultado puede exceder el rango de `char` y al asignarlo de vuelta a `cx` se produce un *truncamiento*.

3.2 Conversiones aritméticas usuales

Cuando los dos operandos tienen tipos distintos, el compilador aplica la siguiente jerarquía (de menor a mayor “rango”):

`char` → `short` → `int` → `long` → `long long` → `float` → `double` → `long double`

El operando de menor rango se convierte al tipo del operando de mayor rango. Por ejemplo, si `cx` es `float` y `cy` es `int`, la suma se realiza en `float`.

3.3 Truncamiento y desbordamiento

Al asignar un valor a un tipo más pequeño, se produce *truncamiento*: se descartan los bits altos. Esto explica por qué en el código original, cuando `cx` y `cy` son `char`, la mayoría de los resultados se “pierden” y muestran caracteres extraños.

Advertencia

El desbordamiento de enteros con signo es **comportamiento indefinido** en C. El desbordamiento de enteros sin signo es *definido* (módulo 2^n). Por eso conviene usar `unsigned` cuando se esperan valores grandes.

4 Operadores de la práctica

La práctica usa las siguientes categorías de operadores:

1. **Aritméticos**: `+`, `-`, `*`, `/`, `%`, `++`, `-`, unarios `+` y `-`.
2. **Relacionales**: `>`, `>=`, `<`, `<=`, `==`, `!=`.
3. **Lógicos**: `!`, `&&`, `||`.
4. **De bits**: `~`, `&`, `|`, `^`, `<<`, `>>`.
5. **De asignación compuesta**: `+=`, `-=`, `*=`, `/=`, `%=`, `<<=`, `>>=`, `&=`, `^=`, `|=`.

A continuación se analiza cada tipo de variable según la tabla de la práctica.

5 Análisis por tipo de dato

5.1 Caso 1: `char` para `cx` y `cy`

```
1 char cx = rand() % LIMITE;  
2 char cy = rand() % LIMITE;  
3
```

Características

- Tamaño: 1 byte.
- Rango con signo: -128 a 127 (plataformas típicas).
- Rango sin signo (`unsigned char`): 0 a 255 .
- `rand() % 256` produce valores 0 a 255 , muchos de los cuales **no caben** en un `char` con signo.

Comportamiento

- **Aritméticos**: todos funcionan porque `char` se promociona a `int`. Pero al reasignar a `cx` se trunca.
- **Módulo %**: funciona porque tras la promoción son `int`.
- **Bits**: funcionan tras la promoción a `int`.
- **%c**: muestra el carácter correspondiente al byte.
- **%d con (int)cx**: muestra el valor numérico con signo.

Problema típico

Si `rand() % 256` da 200 y se asigna a un `char` con signo, el valor almacenado es `-56`. Al imprimir con `%d` verás `-56`, no 200. Esto es clave para entender los resultados de la práctica.

5.2 Caso 2: short para cx y cy

```
1  short cx = rand() % LIMITE;  
2  short cy = rand() % LIMITE;  
3
```

Características

- Tamaño: 2 bytes.
- Rango: `-32,768` a `32,767`.
- Todos los valores de `rand() % 256` caben sin problema.

Comportamiento

- Todos los operadores funcionan.
- No hay desbordamiento en las operaciones básicas.
- El casteo (`short`) tras cada operación es redundante pero válido.

5.3 Caso 3: int para cx y cy

```
1  int cx = rand() % LIMITE;  
2  int cy = rand() % LIMITE;  
3
```

Características

- Tamaño: 4 bytes (típico).
- Rango: $\pm 2,1 \times 10^9$.
- Es el tipo “natural” de la máquina.

Comportamiento

- Todos los operadores funcionan sin promociones adicionales.
- Los casteos (`int`) son totalmente redundantes.
- Es el caso más limpio y predecible.

5.4 Caso 4: `long int` para `cx` y `cy`

```
1  long int cx = rand() % LIMITE;  
2  long int cy = rand() % LIMITE;  
3
```

Características

- `long int` es sinónimo exacto de `long`.
- Tamaño: 8 bytes en Linux x86-64, 4 en Windows.
- Rango: al menos $\pm 2,1 \times 10^9$; típicamente $\pm 9,2 \times 10^{18}$.

Comportamiento

- Idéntico a `int` pero con mayor rango.
- El formato correcto para imprimir sería `%ld`.
- Los casteos (`long int`) son redundantes.

5.5 Caso 5: `long` para `cx` y `cy`

```
1  long cx = rand() % LIMITE;  
2  long cy = rand() % LIMITE;  
3
```

Características

- `long` es exactamente lo mismo que `long int`.
- No hay ninguna diferencia técnica entre ambos.

Nota

Escribir `long` o `long int` es cuestión de estilo. El compilador los trata idénticamente. Lo mismo ocurre con `short` y `short int`, o `unsigned` y `unsigned int`.

5.6 Caso 6: `long long` para `cx` y `cy`

```
1  long long cx = rand() % LIMITE;  
2  long long cy = rand() % LIMITE;  
3
```

Características

- Introducido en C99.
- Tamaño: al menos 8 bytes (64 bits).
- Rango mínimo: $\pm 9,2 \times 10^{18}$.
- Formato `printf`: `%lld`.

Comportamiento

- Todos los operadores funcionan.
- El resultado de las operaciones es `long long`.
- Es el tipo entero más grande estándar.

5.7 Caso 7: `float` para `cx` y `cy`

```
1  float cx = rand() % LIMITE;  
2  float cy = rand() % LIMITE;  
3
```

Características

- Tamaño: 4 bytes.
- Precisión: ≈ 7 dígitos decimales significativos.
- Representación IEEE 754: 1 bit signo, 8 exponente, 23 mantisa.

Problemas graves

Este es el caso que **no compila tal cual**. Los siguientes operadores **no están definidos para float**:

- `%` (módulo) $\rightarrow$ usar `fmod()` de `<math.h>`.
- `~, &, |, ^, <<, >>` (bits).

- `%=`, `<<=`, `>>=`, `&=`, `^=`, `|=`.
- `%c` con un `float` es comportamiento indefinido.

Solución

Convertir a entero antes de aplicar esos operadores, o eliminar esas secciones. Por ejemplo:

```
1  #include <math.h>
2  cx = fmod(cx, cy);           // en lugar de cx % cy
3  cx = (float)(~(int)cy);      // bits, pero pierde el sentido de "float"
4
```

5.8 Caso 8: double para cx y cy

```
1  double cx = rand() % LIMITE;
2  double cy = rand() % LIMITE;
3
```

Características

- Tamaño: 8 bytes.
- Precisión: ≈ 15 dígitos decimales significativos.
- Formato `printf`: `%lf` o `%f`.

Comportamiento

Presenta exactamente los mismos problemas que `float` con los operadores de bits y módulo. La única diferencia es la precisión.

5.9 Caso 9: long double para cx y cy

```
1  long double cx = rand() % LIMITE;
2  long double cy = rand() % LIMITE;
3
```

Características

- Tamaño: 10, 12 o 16 bytes según la plataforma.
- Precisión: al menos la de `double`, típicamente mayor.
- Formato `printf`: `%Lf` (con L mayúscula).

Comportamiento

Idéntico a `double` en cuanto a operadores permitidos: los de bits y módulo no funcionan. El código de la práctica usa `%lf` para imprimir `long double`, lo cual es **incorrecto**: debería ser `%Lf`.

6 Tabla resumen de compatibilidad

Operador	Enteros	float	double	long double
+ - * /	✓	✓	✓	✓
%	✓	×	×	×
++ -	✓	✓	✓	✓
+ - (unario)	✓	✓	✓	✓
> >= < <=	✓	✓	✓	✓
== !=	✓	✓	✓	✓
! &&	✓	✓	✓	✓
~	✓	×	×	×
& ^	✓	×	×	×
<< >>	✓	×	×	×
+= -= *= /=	✓	✓	✓	✓
%=	✓	×	×	×
<<= >>=	✓	×	×	×
&= ^= =	✓	×	×	×

7 El problema del%c y los tipos no-int

En el código de la práctica se usa abundantemente `printf("%c", cx)`. El especificador `%c` espera un argumento de tipo `int`. Cuando se le pasa un `char`, `short` o `int`, no hay problema porque las promociones de `printf` lo manejan. Pero cuando se le pasa un `long`, `long long`, `float`, `double` o `long double`, se produce **comportamiento indefinido**.

Regla práctica

Si quieres imprimir el carácter correspondiente al byte bajo de una variable de cualquier tipo entero, castea explícitamente:

```
1 printf("%c", (int)cx);
2
```

Si la variable es de punto flotante, primero conviértela a entero:

```
1 printf("%c", (int)cx);
2
```

8 El problema del truncamiento en la práctica

Una de las cosas más confusas al ejecutar `operadorconchar.c` es que los resultados parecen “aleatorios” o “sin sentido”. Esto se debe a que cada operación se asigna de vuelta a `cx`, que es de tipo `char`, y por tanto se trunca al rango -128 a 127 .

Por ejemplo, si `cx = 200` y `cy = 100`:

```
1  cx = (char)(cx * cy);    // 20000 -> truncado a char
2
```

$20000 \bmod 256 = 32$, pero como `char` tiene signo, si el bit alto está activo el valor se interpreta como negativo. El resultado real es -32 .

Este fenómeno se repite en cada operación, y por eso los valores se van “degradando” de forma aparentemente caótica. Con `short`, `int`, `long`, etc., el truncamiento es menos frecuente o inexistente, y los resultados son más predecibles.

9 Recomendaciones de codificación

1. **No usar `char` con signo para almacenar valores > 127 .** Usar `unsigned char` o un tipo mayor.
2. **No usar operadores de bits ni % con `float`, `double` o `long double`.**
3. **Castear siempre a `int` al usar %c** con tipos mayores que `int`.
4. **Usar el formato correcto** en `printf`: `%hhd` para `char`, `%hd` para `short`, `%d` para `int`, `%ld` para `long`, `%lld` para `long long`, `%f` para `float`, `%lf` para `double`, `%Lf` para `long double`.
5. **No asumir tamaños:** usar `sizeof` y los tipos de `<stdint.h>` (`int8_t`, `int16_t`, `int32_t`, `int64_t`) cuando la portabilidad importe.

10 Conclusión

La práctica `operadorcon*.c` es un excelente ejercicio para comprender cómo el tipo de dato afecta radicalmente el comportamiento de los operadores en C. Los tipos enteros pequeños (`char`, `short`) sufren promociones y truncamientos que producen resultados contraintuitivos. Los tipos enteros grandes (`int`, `long`, `long long`) se comportan de forma más predecible. Los tipos de punto flotante (`float`, `double`, `long double`) simplemente **no admiten** los operadores de bits ni el módulo, lo que obliga a reescribir parte del código o a eliminar esas secciones.

Comprender estas diferencias es fundamental para escribir código C correcto, portable y sin comportamientos indefinidos.

Propiedad y uso

Este documento es propiedad de **Tec. Emilio C. Vargas** y se realizó para el **Laboratorio Virtual de Programación** disponible en <https://proyectos.emiliocolor.com>. Su uso está destinado a fines educativos dentro de dicho laboratorio.